

Día 1 - Entrada Secreta

1. Introducción y Problema El problema plantea simular una caja fuerte con un dial circular (0-99). Se empieza en 50; si vas a la izquierda (L10) restas, y si vas a la derecha (R10) sumas. El reto tiene dos partes que cambian la regla de cómo se obtiene la contraseña:

- **Parte A:** La contraseña depende de **dónde termina** el dial tras cada rotación: suma 1 cada vez que un giro acaba exactamente en 0.
- **Parte B:** La contraseña cuenta **cuántas veces el dial pasa por el cero** durante el giro (no solo dónde acaba, sino cada cruce intermedio).

Ambas partes leen la misma entrada y comparten toda la mecánica del dial; lo único que cambia es la regla de puntuación. Esa observación es la que guía el diseño: si lo único que varía es una regla, esa regla debe poder enchufarse desde fuera sin tocar el resto.

2. Arquitectura por capas He reorganizado el día en **cuatro capas**, de modo que las dependencias apuntan siempre hacia el dominio y nunca al revés:

```
software.ulpgc.aoc.day01
├── model          (dominio puro, no depende de nadie)
│   ├── Dial
│   ├── Instruction
│   └── SecurityProtocol
├── io             (frontera de entrada)
│   └── InstructionLoader
├── control        (orquesta el caso de uso)
│   ├── Safe
│   └── SecurityProtocols
└── application    (detalles y arranque)
    ├── ResourceInstructionLoader
    └── a/Main01a, b/Main01b
```

Dirección de dependencias: application → control → (io + model) y io → model. El dominio (model) no importa nada del proyecto, así que es el centro estable del sistema.

3. Explicación clase a clase Capa model (dominio puro)

- **Dial** (*record immutable*): representa la rueda y su única tarea es la aritmética circular. Su constructor normaliza siempre la posición al rango 0-99, y rotate(int) devuelve un **nuevo** Dial en vez de mutar el actual. No sabe nada de ficheros ni de reglas de puntuación → **alta cohesión**.
- **Instruction** (*record*): representa una orden de giro ("L10", "R48"). Concentra el parseo y la validación en of(String), que devuelve un Optional y descarta entradas inválidas (nulos, vacíos, basura). Expone movement() (L resta, R suma). Sacar esto de la caja fuerte es lo que le da a Safe una sola responsabilidad.
- **SecurityProtocol** (*interfaz funcional*): la **abstracción** de la regla de puntuación (calculatePoints(oldDial, movement, newDial)). Vive en el dominio porque solo habla el idioma del dominio (Dial) y no depende de ninguna otra capa. Es la pieza que permite el DIP.

Capa io (frontera de entrada)

- **InstructionLoader** (*interfaz*): define *qué* se necesita de la entrada (`List<Instruction> loadAll()`) sin decir *cómo* se obtiene. El control y el arranque dependen de esta abstracción, no del detalle de lectura.

Capa control (orquesta el caso de uso)

- **Safe**: el corazón del caso de uso. Mantiene el estado (dial actual y contador) y, por cada instrucción, gira el dial y delega la puntuación en el `SecurityProtocol` inyectado. No lee ficheros ni parsea texto: recibe `Instruction` ya validadas. `rotate(String)` se conserva por comodidad/robustez y delega en `Instruction.of`.
- **SecurityProtocols**: agrupa las dos estrategias concretas (`PART_A`, `PART_B`) como constantes reutilizables. Añadir una regla nueva es crear otra constante, sin tocar `Safe`.

Capa application (detalles y arranque)

- **ResourceInstructionLoader** (*implements InstructionLoader*): el detalle de bajo nivel; lee las líneas de un recurso del classpath y las convierte en `Instruction`. Es intercambiable (otra fuente de datos = otra implementación).
- **Main01a / Main01b** (*composition root*): el único punto donde se eligen las piezas concretas (qué cargador y qué protocolo) y se conectan por inyección. La Parte A y la Parte B se diferencian solo en la estrategia inyectada.

4. Principios y diseños aplicados

- **Inversión de Dependencias (DIP)**: `Safe` depende de la abstracción `SecurityProtocol`, no de la lógica concreta de A o B; el arranque depende de `InstructionLoader`, no de cómo se leen los datos.
- **Abierto/Cerrado (OCP)**: añadir una regla es inyectar otra estrategia; `Safe` queda cerrada a modificación y abierta a extensión por polimorfismo.
- **Responsabilidad Única (SRP)**: `Dial` solo hace aritmética, `Instruction` solo parsea/valida, `Safe` solo orquesta, `ResourceInstructionLoader` solo lee I/O.
- **Segregación de Interfaces (ISP)**: `InstructionLoader` expone un único método cohesivo.
- **Inyección de Dependencias (DI) / Patrón Strategy**: el protocolo y el cargador se pasan desde fuera; el comportamiento se elige sin tocar el núcleo.
- **Bajo Acoplamiento y Alta Cohesión**: las capas se comunican por abstracciones y cada clase agrupa lo que está estrechamente relacionado.
- **DRY**: la lectura de entrada está centralizada en una sola implementación reutilizable.
- **Tell, Don't Ask / encapsulamiento**: se le ordena a `Safe` que aplique una instrucción (`apply`) en vez de preguntarle su estado para decidir fuera; el `Dial` es inmutable y normaliza en su propio constructor.

5. Conclusión La separación en capas mantiene el dominio en el centro y empuja los detalles (I/O, arranque) a los bordes. El resultado es un sistema con bajo acoplamiento, fácil de probar (los tests instancian `Safe` con la estrategia que quieran, sin tocar ficheros) y preparado para crecer: cambiar la fuente de datos o añadir una nueva regla no obliga a modificar el núcleo. Se verificó que el comportamiento se conserva: ambas partes y todos los casos de prueba (Parte A = 3, Parte B = 6 sobre el ejemplo) siguen dando el mismo resultado tras la refactorización.